

Diagrams as Code

An introduction to PlantUML and Mermaid, draw your UML by typing it

```
@startuml
Detective -> Witness : "Did you see anything?"
Witness --> Detective : "...maybe."
@enduml
```

PlantUML

```
graph LR
  A[Idea] --> B[Text]
  B --> C[Diagram]
```

Mermaid

Warm-up: how well do you know JSON?

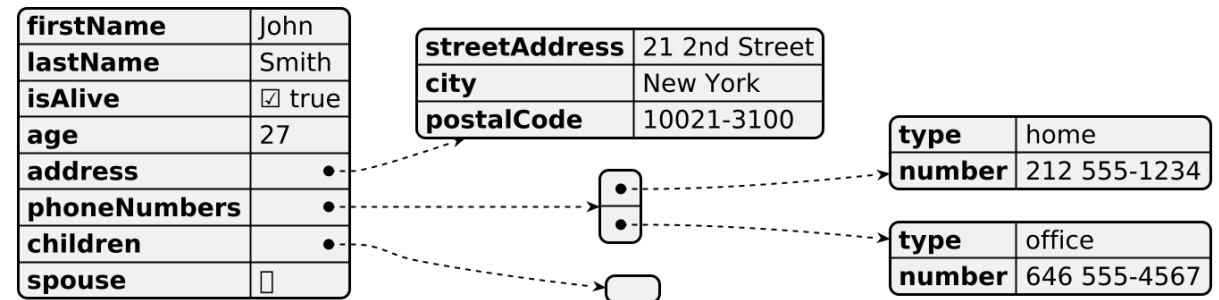
PAIR UP · 1 MIN

30 seconds with your neighbour: how would you DRAW this data?

```
{
  "firstName": "John",
  "lastName": "Smith",
  "isAlive": true,
  "age": 27,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "postalCode": "10021-3100"
  },
  "phoneNumbers": [
    { "type": "home", "number": "212 555-1234" },
    { "type": "office", "number": "646 555-4567" }
  ],
  "children": [],
  "spouse": null
}
```

the JSON itself

Somebody fed this JSON to a program and got a diagram. **Guess: which program, and how many lines of 'drawing code' did it take?**



Output of the public PlantUML server (real render)

Answer: 2 lines of wrapper - **@startjson** / **@endjson**, around the JSON you already know.

By the end of this hour you can...

- 1 **Render** a diagram in the online PlantUML editor and in mermaid.live, nothing installed
- 2 **Write** basic PlantUML: sequence, class and use-case diagrams from a written spec
- 3 **Write** basic Mermaid: flowchart, sequence and class diagrams from a written spec
- 4 **Debug** a broken diagram: spot missing wrappers, wrong arrows, unquoted labels
- 5 **Decide & proceed** choose the right tool and plug diagrams into git + your report workflow



Why diagrams as code?

Motivation before mechanics

- ▶ The 2 a.m. problem with drawn diagrams
- ▶ What changes when a diagram is a text file
- ▶ Meet PlantUML (2009) and Mermaid (2014)

The 2 a.m. problem

A diagram drawn in a drawing tool is a picture. A picture cannot be merged, diffed or reviewed.

Drawn diagrams (picture files)

- Change one class name → reopen the tool, find the box, retype, re-align, re-export
- Two teammates edit the same diagram → one overwrites the other
- `report_final_v3_REAL_updated.drawio` - which one is in the submission?
- A reviewer sees only the exported image, never the reasoning

Typed diagrams (text files)

- Rename = edit one line of text; the layout re-solves itself
- Git merges, diffs and blames diagrams exactly like code
- Pull-request review: the change is visible as a text diff
- Email it, paste it in chat, template it, generate it in CI
- Accessible route in for UDL: diagrams communicated as code

Two tools, one idea

Both are free, open source, and render text → diagram. They differ in engine, coverage and ecosystem.

PlantUML

- **First released** 17 April 2009 by Arnaud Roques
- **Engine:** Java, runs as a .jar, in IDEs, or on a web server
- **License:** GPL, open source
- **Coverage:** every UML diagram type, plus JSON/YAML, Gantt, mindmap, ER, wireframes...
- **Home:** plantuml.com

Mermaid

- **Created** 2014 by Knut Sveidqvist
- **Engine:** JavaScript, renders in the browser, no server needed
- **License:** MIT, open source
- **Coverage:** flowcharts, sequence, class, state, ER, Gantt, pie, gitGraph, mindmap...
- **Home:** mermaid.js.org · editor: mermaid.live

Family note: they belong to the 'diagrams-as-code' family with Graphviz/DOT and D2; aggregators like kroki.io render many of them through one URL.



PlantUML

The UML workhorse

- ▶ Tour of the public web editor (nothing to install)
- ▶ Anatomy of a file: wrappers, declarations, arrows, labels
- ▶ Sequence · class · use-case syntax with live renders
- ▶ Bug hunt #1

PlantUML in 60 seconds

What it is, where it runs, what comes out

- **Idea:** you write plain text between wrappers; a renderer lays out the diagram for you
- **Runs:** public web server (no install), local plantuml.jar (needs a JVM), IDE plugins (VS Code, IntelliJ...), GitLab integration
- **Outputs:** PNG, SVG, LaTeX, and even ASCII art, same text, many formats
- **Diagrams:** sequence, use case, class, object, activity, component, deployment, state, timing
- **Bonus:** JSON/YAML, Gantt, mindmap, WBS, ER, network (nwdiag), wireframes (salt), Archimate, regex, EBNF, AsciiMath maths
- **Styling:** themes (!theme plain, sketchy, superhero...) and skinparam knobs

Try it now (30 s):

1. open plantuml.com/plantuml
2. delete the sample text
3. type:

```
@startuml
    A -> B : hi
@enduml
```
4. press Submit
5. change hi to hello -
Submit again

Tour: the PlantUML web editor

Two official front doors + one URL trick worth understanding

- **Classic server page:** www.plantuml.com/plantuml, text area, Submit button, theme picker, PNG below
- **New web editor:** editor.plantuml.com, split view with live preview
- **Output links:** every diagram has PNG / SVG / ASCII-Art URLs
- **The URL trick:** the gibberish after `/png/` is your diagram, compressed and encoded, the URL IS the file. Paste it to a teammate and they see your diagram.
- **Errors:** syntax mistakes come back as an image with the offending line highlighted, we will use that in the bug hunt



scan → www.plantuml.com/plantuml

No laptop? phone browser works - the server is just a web form.

Live demo 1: first real diagram

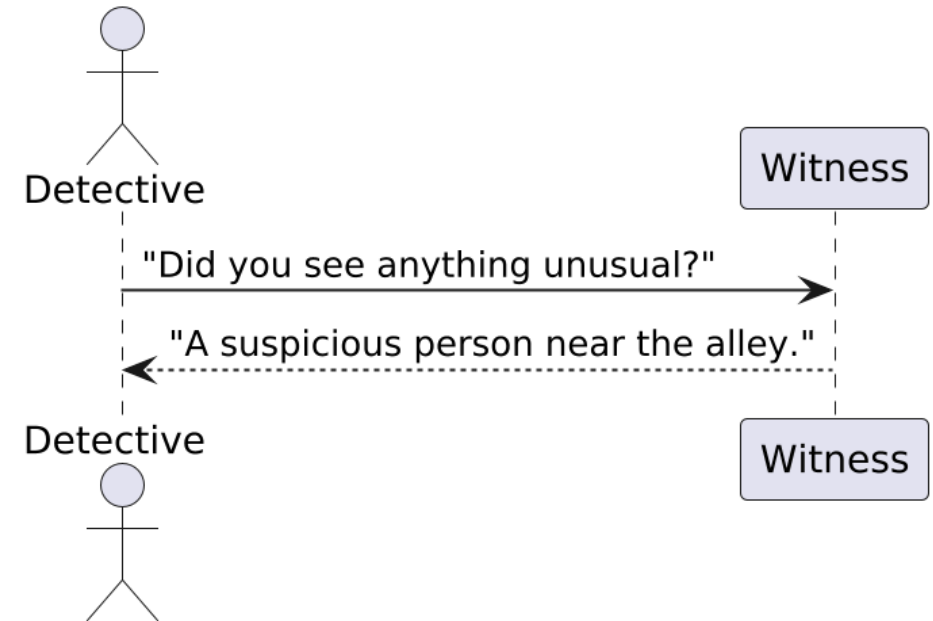
Instructor types, you type along, then we break it on purpose

```
@startuml
actor Detective
participant Witness

Detective -> Witness : "Did you see anything unusual?"
Witness --> Detective : "A suspicious person near the alley."
@enduml
```

PlantUML

Demo script: (1) submit as-is; (2) add a third participant Lab and a message to it, watch the lifeline appear; (3) delete the @ sign of @enduml and submit: read the error image together; (4) undo.



server output for exactly this source

Reading the arrows: A -> B solid = call/message, A --> B dashed = reply. Labels go after a colon.

Anatomy of a PlantUML file

Four ideas cover 90% of what you will type

```
@startuml                                ' 1 wrapper
actor Detective                          ' 2 declaration
participant Witness as w                ' 3 alias
Detective -> w : "Question?"            ' 4 relation + label
w --> Detective : "Answer."            ' (dashed = reply)
@enduml                                 ' 1 wrapper
```

1 Wrapper - @startuml / @enduml (or @startjson, @startgantt, @startmindmap...). Missing wrapper = the classic first error.

2 Declaration - actor, participant, class, usecase... tells the renderer what kind of thing you mean; first use can also declare implicitly.

3 Alias - 'as w' gives a short handle so lines stay readable; the long name still shows in the picture.

4 Relation + label - the arrow draws the line; text after : becomes the label. Quote labels that contain spaces or odd characters.

' Comments - a line starting with ' is ignored, annotate your diagrams like code.

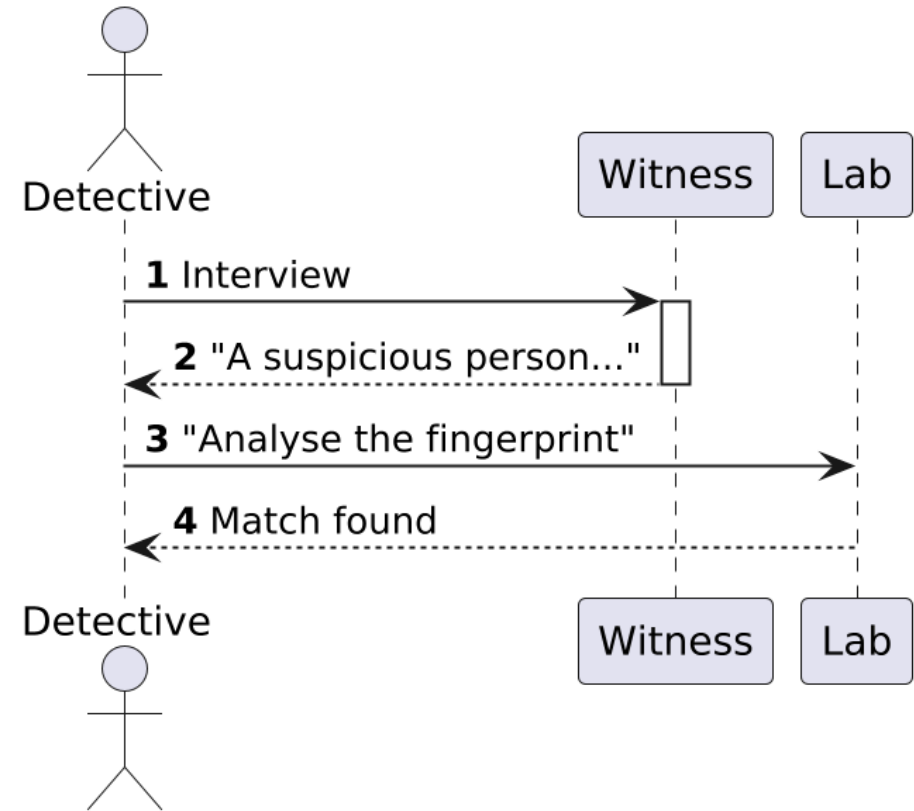
Sequence diagrams: the syntax that matters

Messages, replies, numbering, activation bars

```
@startuml
autonumber
actor Detective
participant Witness
participant Lab

Detective -> Witness : Interview
activate Witness
Witness --> Detective : "A suspicious person..."
deactivate Witness
Detective -> Lab : "Analyse the fingerprint"
Lab --> Detective : Match found
@enduml
```

PlantUML



autonumber + activate rendered by the server

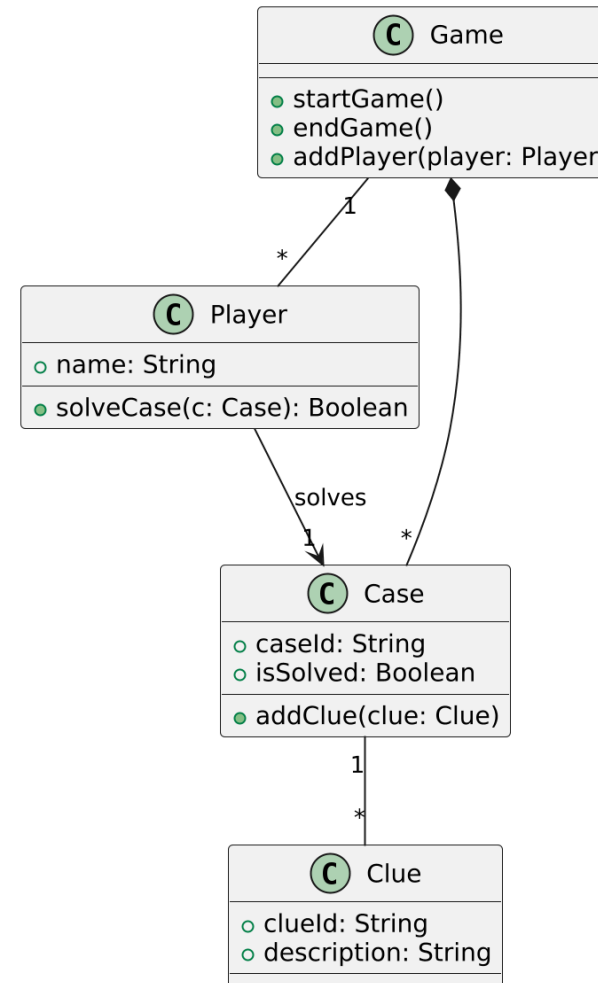
autonumber numbers the messages (great for report references); **activate X / deactivate X** draws the focus bar on a lifeline; **actor** vs **participant** = stick figure vs box.

Class diagrams: members, visibility, relations

The diagram you will submit most often in this course

```
@startuml
class Game {
+startGame()
+endGame()
+addPlayer(player: Player)
}
class Player {
+name: String
+solveCase(c: Case): Boolean
}
class Case {
+caseId: String
+isSolved: Boolean
+addClue(clue: Clue)
}
class Clue {
+clueId: String
+description: String
}
Game *-- "*" Case
Game "1" -- "*" Player
Case "1" -- "*" Clue
Player --> "1" Case : solves
@enduml
```

PlantUML



Relation cheat-sheet

A -- B link
A <|-- B inherits
A *-- B composition
A o-- B aggregation
A ..> B depends
A ..|> B realizes

"1" -- "*" multiplicities
in quotes

Visibility: + public - private # protected ~ package

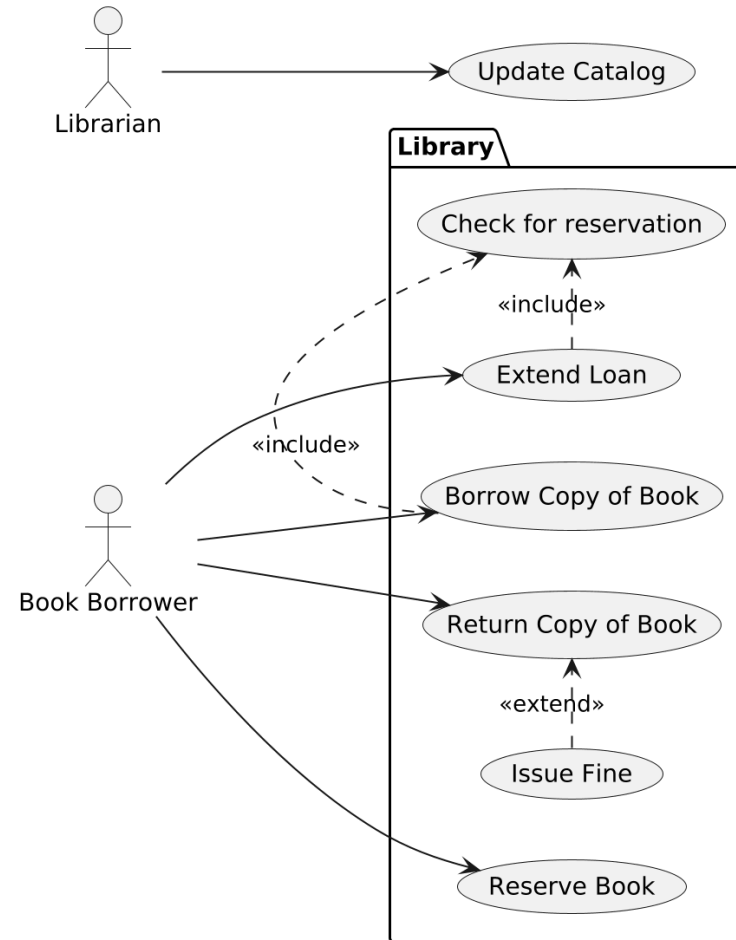
Use-case diagrams: actors, ellipses, include/extend

Same five ideas, new keywords

```
@startuml
left to right direction
package Library {
    usecase "Reserve Book" as UC1
    usecase "Borrow Copy of Book" as UC2
    usecase "Return Copy of Book" as UC3
    usecase "Extend Loan" as UC4
    usecase "Check for reservation" as UC9
    usecase "Issue Fine" as UC10
}
actor "Book Borrower" as bb
actor "Librarian" as ln
usecase "Update Catalog" as UC8

bb --> UC1
bb --> UC2
bb --> UC3
bb --> UC4
ln --> UC8
UC4 .> UC9 : <<include>>
UC2 .> UC9 : <<include>>
UC10 .> UC3 : <<extend>>
@enduml
```

PlantUML



actor

stick figure

usecase ".."

or (Name)

package { }

groups them

A .> B :

<<include>>

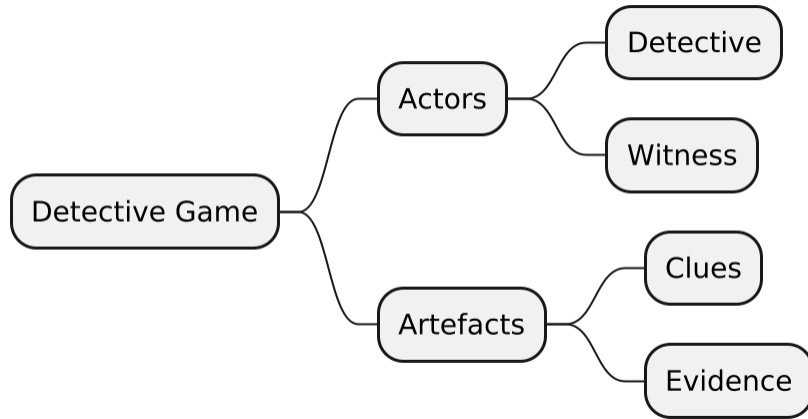
<<extend>>

dashed, always with
the stereotype

PlantUML is not only UML

The same wrapper idea, other diagram languages

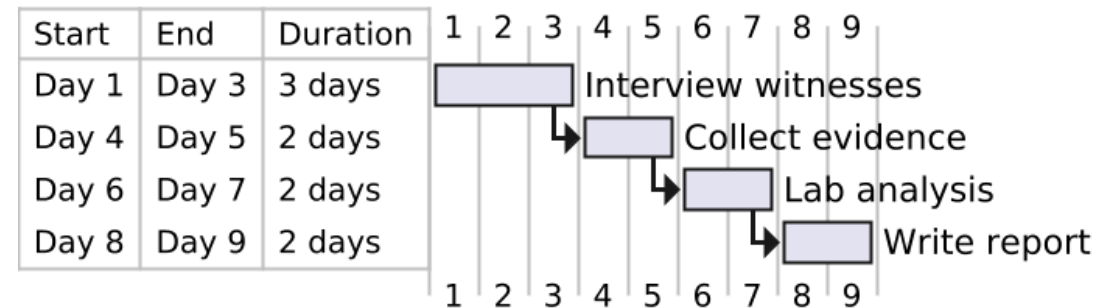
Also built in: JSON/YAML views, WBS, ER (IE syntax), network (nwdiag), salt wireframes, Archimate, timing, SDL, regex, EBNF, AsciiMath/JLaTeX maths.



@startmindmap ... @endmindmap

```
@startmindmap
* Detective Game
** Actors
*** Detective
*** Witness
** Artefacts
*** Clues
*** Evidence
@endmindmap
```

Case investigation plan



@startgantt ... @endgantt

```
@startgantt
title Case investigation plan
[Interview witnesses] lasts 3 days
[Collect evidence] starts at [Interview witnesses]'s end and lasts 2 days
[Lab analysis] starts at [Collect evidence]'s end and lasts 2 days
[Write report] starts at [Lab analysis]'s end and lasts 2 days
@endgantt
```

Bug hunt #1: this diagram will not render right

```
startuml
left to right direction
actor Detective
actor Witness
actor Suspect
rectangle "Detective Game System" {
    Detective --> (Gather Clues)
    Detective --> (Analyze Evidence)
    Detective --> (Solve Case)
    Witness --> (Provide Testimony)
    Witness --> (Answer Questions)
    Suspect --> (Provide Alibi)
    Suspect --> (Answer Questions)
    Suspect --> (Evade Questioning)
    (Gather Clues) --> (Analyze Evidence) : <<include>>
    (Analyze Evidence) --> (Solve Case) : <<extend>>
    (Solve Case) ..> (Gather Clues) : <<uses>>
    (Solve Case) ..> (Analyze Evidence) : <<uses>>
}
@enduml
```

Hunting checklist

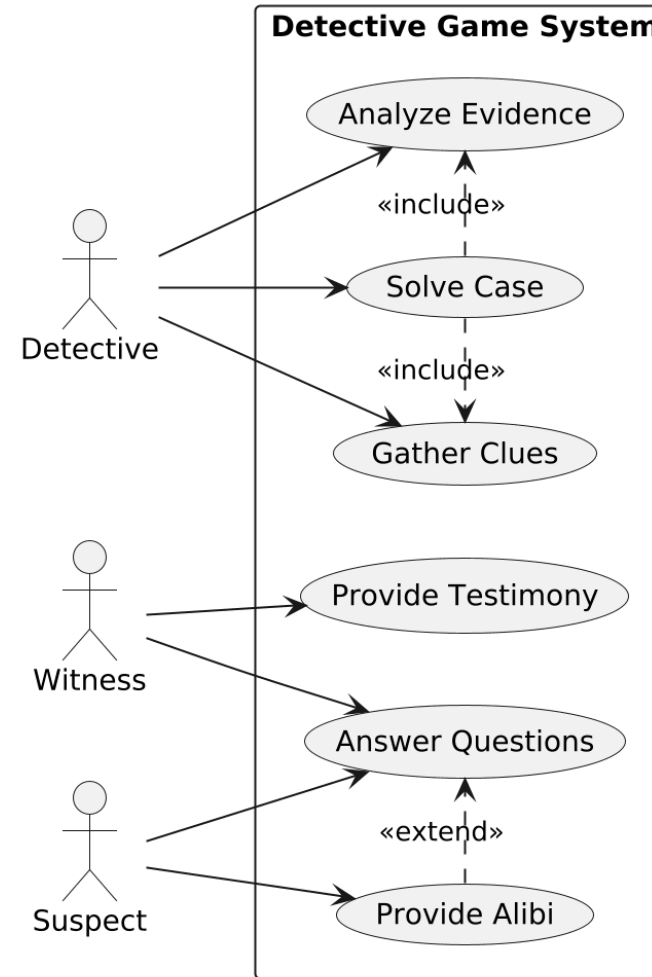
- Does the file even start correctly?
- Which arrows should be dependencies?
- Which way must an extend arrow point?
- Are all stereotypes real UML?

Write your 4 answers down - we compare in 3 minutes.

Bug hunt #1: the fixed version

Four edits - and the server is happy again

```
@startuml
left to right direction
actor Detective
actor Witness
actor Suspect
rectangle "Detective Game System" {
    Detective --> (Gather Clues)
    Detective --> (Analyze Evidence)
    Detective --> (Solve Case)
    Witness --> (Provide Testimony)
    Witness --> (Answer Questions)
    Suspect --> (Answer Questions)
    Suspect --> (Provide Alibi)
    (Solve Case) .> (Gather Clues) : <<include>>
    (Solve Case) .> (Analyze Evidence) : <<include>>
    (Provide Alibi) .> (Answer Questions) : <<extend>>
}
@enduml
```



- 1 @ added
- 2 dashed .>
- 3 extend
re-aimed
- 4 <<uses>>
gone

Bug hunt #2: this diagram will render but not right

```
@startuml
class Library {
+ name: String
+ addBook(Book b)
}
class Book {
+ title: String
+ author: String
+ borrow()
}
class EBook {
+ fileSize: double
+ format: String
}

Library "1" -- "0..*" Book
EBook -- Book
@enduml
```

Hunting checklist

- Is the relationship correctly identified?

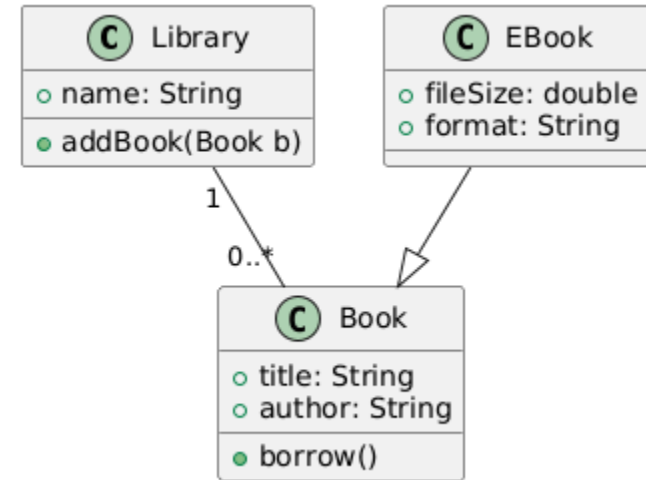
Bug hunt #2: the fixed version

Four edits - and the server is happy again

```
@startuml
class Library {
+ name: String
+ addBook(Book b)
}
class Book {
+ title: String
+ author: String
+ borrow()
}
class EBook {
+ fileSize: double
+ format: String
}

Library "1" -- "0..*" Book
EBook --|> Book

Library "1" -- "0..*" Book
EBook --|> Book
' <-- Fixed inheritance arrow
@enduml
```





Mermaid

The markdown-native one

- ▶ mermaid.live: code left, picture right, zero waiting
- ▶ Flowcharts, sequence, class, and a gallery of extras
- ▶ Why your GitHub README already speaks Mermaid
- ▶ Bug hunt #3

Mermaid in 60 seconds

Markdown-inspired text, rendered by JavaScript in your browser

- **Idea:** the FIRST LINE names the diagram type; the rest declares nodes and links, no wrapper tags
- **Runs:** in the browser (mermaid.live), in Markdown viewers, in Node.js / mermaid-cli for CI
- **Native homes:** GitHub Markdown (since Feb 2022), GitLab, and note apps such as Obsidian and Notion
- **Diagrams:** flowchart, sequence, class, state, ER, Gantt, pie, gitGraph, journey, timeline, mindmap, C4 and more
- **Comments:** %% (two percent signs) - not the single quote of PlantUML
- **Exports:** SVG / PNG from the live editor; shareable editor links

Try it now (30 s):

1. open `mermaid.live`
2. type:

```
graph LR
  A[tea] --> B{milk?}
  B -- yes --> C[chai]
```
3. watch it draw while you type
4. change LR to TD

Tour: the Mermaid Live Editor

mermaid.live - the fastest feedback loop of the two tools

- **Split view:** code on the left, diagram on the right, re-rendered on every keystroke
- **Config pane:** theme and renderer options as a small JSON block
- **Export:** SVG and PNG buttons; SVG is the one you want in reports (scales, tiny)
- **Share:** a link that carries your diagram in the URL - same trick as PlantUML
- **Examples menu:** a gallery of every diagram type - the best syntax textbook there is
- **Errors:** a parse-error panel tells you the line and what it expected - read it, do not fear it



scan → mermaid.live

Offline later: the mermaid JS library also runs from any local page or Node CLI.

Live demo 2: flowcharts & node shapes

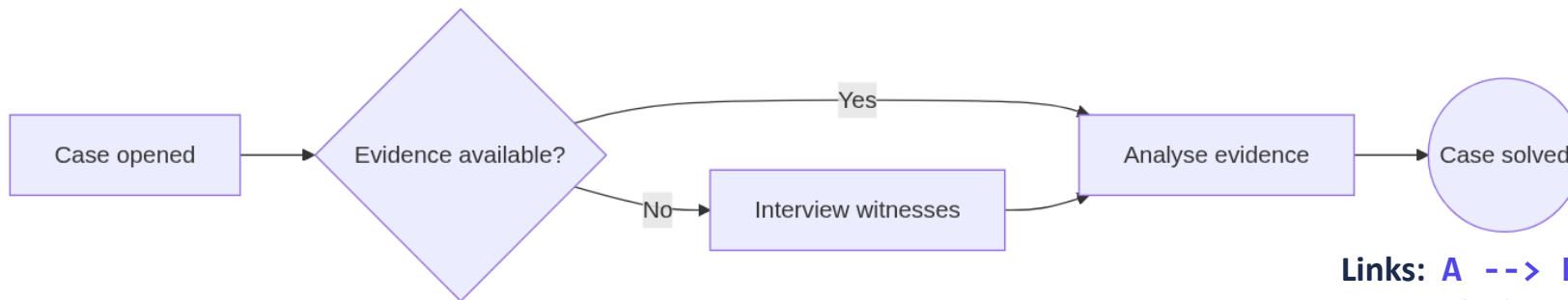
LIVE DEMO · 4 MIN

Brackets are shapes: [] rectangle, { } diamond, (()) circle, ([]) stadium

flowchart LR

```
A[Case opened] --> B{Evidence available?}
B -- Yes --> C[Analyse evidence]
B -- No --> D[Interview witnesses]
D --> C
C --> E((Case solved))
```

Demo script: (1) type it, preview follows each line; (2) flip LR → TD and watch the whole diagram rotate; (3) change one -- Yes --> label; (4) add E --> F[Appeal?] and connect it.



rendered by mermaid.ink from exactly this source

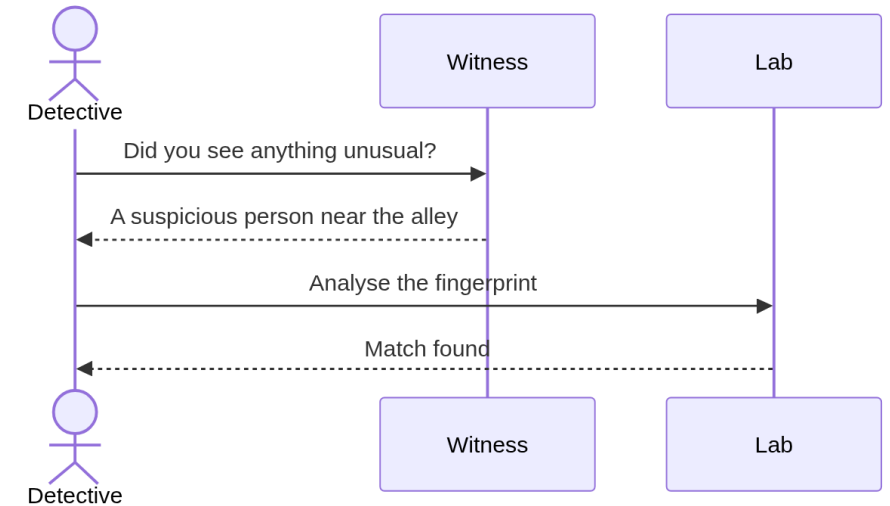
Links: A --> B arrow, A --- B plain, A -.-> B dotted, A ==> B thick;

labels: A -- text --> B or A -->|text| B

Mermaid sequence diagrams

Compare with PlantUML: same concepts, different punctuation

```
sequenceDiagram
    actor Detective
    participant Witness
    participant Lab
    Detective->>Witness: Did you see anything unusual?
    Witness-->>Detective: A suspicious person near the alley
    Detective->>Lab: Analyse the fingerprint
    Lab-->>Detective: Match found
```

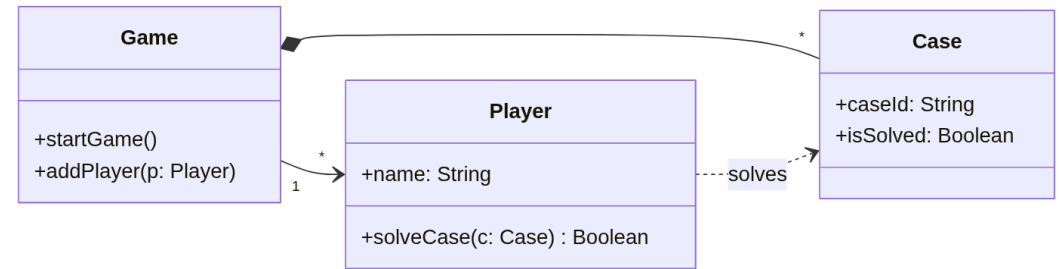


- Arrows: A->>B solid message, A-->>B dashed reply (note the double head)
- actor / participant declare the lifelines, in order of appearance
- Note over A,B: text adds a note bar; loops and alt blocks exist too

Mermaid class diagrams

Members need the colon; relations reuse UML-ish symbols

```
classDiagram
    direction LR
    class Game {
        +startGame()
        +addPlayer(p: Player)
    }
    class Player {
        +name: String
        +solveCase(c: Case) Boolean
    }
    class Case {
        +caseId: String
        +isSolved: Boolean
    }
    Game "1" --> "*" Player
    Game "*" -- "*" Case
    Player ..> Case : solves
```



- class C { +member: Type +method(x: T) R } - visibility signs as in UML
- Relations: C <|-- D inherit, C *-- D composition, C o-- D aggregation, C --> D assoc, C ..> D dependency
- Multiplicities in quotes: Game "1" --> "*" Player; direction LR lays classes side by side

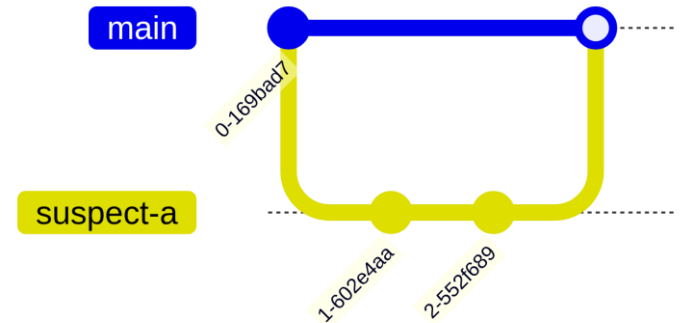
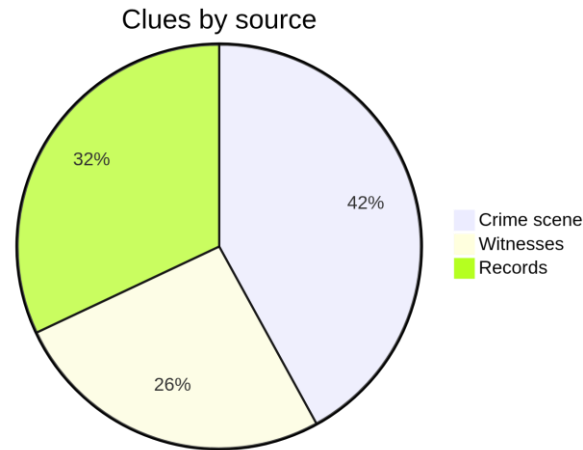
state, pie, git history

Same first-line-keyword habit, new vocabularies

```
stateDiagram-v2
    direction LR
    [*] --> Investigating
    Investigating --> Analysing : evidence found
    Analysing --> Investigating : dead end
    Analysing --> Solved : suspect charged
    Solved --> [*]
```

```
pie title Clues by source
    "Crime scene" : 42
    "Witnesses" : 26
    "Records" : 32
```

```
gitGraph
    commit
    branch suspect-a
    commit
    commit
    checkout main
    merge suspect-a
```



yes - it can draw your git branch history

Also in the box: erDiagram (entity-relationship), gantt, journey, timeline, mindmap, C4, quadrant - all in the live editor's Examples menu.

Mermaid in the wild: your README renders it

GitHub has rendered ``mermaid fences natively since February 2022

```
``mermaid
flowchart LR
    A[Idea] --> B[Text] --> C[Diagram in README]
``
```



what any visitor sees on GitHub / GitLab - no image file at all

- README, issues, pull requests, wikis: the fence just works
- GitLab: Mermaid out of the box; PlantUML via its integration (on for GitLab.com)
- GitHub: PlantUML is NOT rendered natively - export an image or render in CI
- So: a Mermaid diagram travels with your repo forever, versioned with it

Consequence for this course: put Mermaid in the repo README for overview diagrams; keep the full UML set as PlantUML files beside it.

Bug hunt #3: Mermaid edition

PAIR UP · 2 MIN

Three faults, two minutes, same pairs

flowchart TD

A[Start interview] -> B{Alibi given?}

B -- Yes --> C[Check records (police)]

C --> D[Close case

suspect code

Hunting checklist

- Flowchart links: which arrow tokens actually exist?
- Node text with () or other special characters: what does the docs say?
- Count your brackets on the last line...

Bonus: paste it into mermaid.live - what does the error panel say?

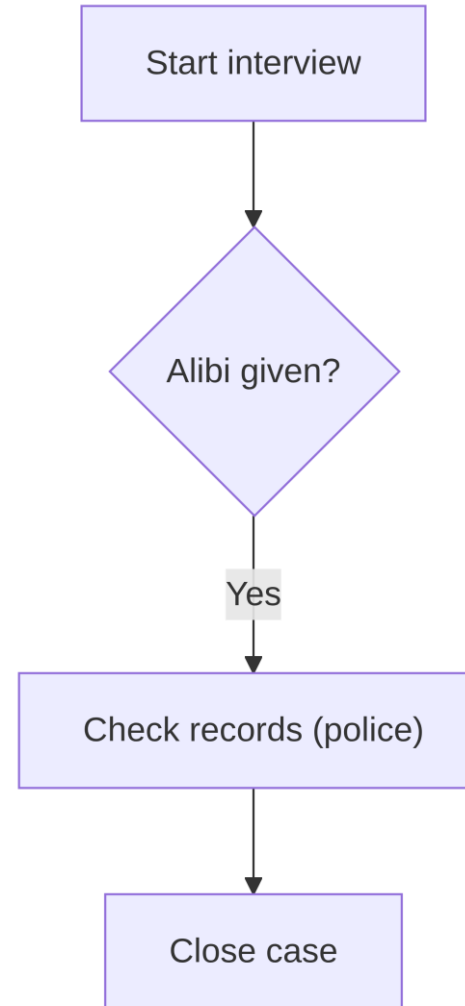
Bug hunt #3: fixed & rendered

Three characters changed - diagram saved

flowchart TD

```
A[Start interview] --> B{Alibi given?}
B -- Yes --> C["Check records (police)"]
C --> D[Close case]
```

Pattern to remember: most Mermaid parse errors are (a) unknown arrow token, (b) unquoted special character, (c) unclosed bracket. Three checks fix the majority of real-world failures.



1 -> became
-->
2 label quoted:
C[".(police)"]
3 D[Close case]
bracket closed



Choosing & proceeding

From toy to tool

- ▶ Head-to-head: what each tool is genuinely better at
- ▶ The workflow: editor → repo → report, and back
- ▶ Course rules for submissions

Head-to-head

	PlantUML	Mermaid
First release	2009 · Arnaud Roques	2014 · Knut Sveidqvist
Engine	Java (JVM): server, jar, IDE plugins	JavaScript: browser, Node, CLI
License	GPL (open source)	MIT (open source)
Zero-install editor	plantuml.com/plantuml · editor.plantuml.com	mermaid.live
GitHub Markdown	not rendered natively	native since Feb 2022
GitLab Markdown	PlantUML integration (on for GitLab.com)	native, out of the box
UML depth	all UML types (incl. deployment, timing, object)	class, sequence, state, ER + flowchart; no deployment/timing
Extras	JSON/YAML, salt wireframes, nwdiag, Archimate...	pie, gitGraph, journey, timeline, C4...
Styling control	skinparam + themes (!theme ...)	themes + init config

Quick poll: which tool would you pick?

POLL · 3 MIN

A

Overview diagram inside your team's GitHub README

Mermaid - renders natively, no image file

B

Full UML set for the design report, incl. deployment view

PlantUML - Mermaid has no deployment diagram

C

Diagram in a GitLab wiki page

either - GitLab speaks both (PlantUML via integration)

D

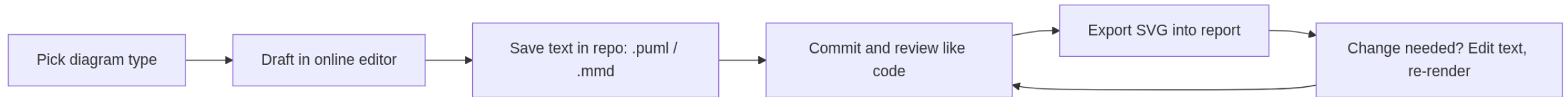
Regenerate every diagram from CI on each commit

either - plantuml.jar and mermaid-cli both run headless

Rule of thumb: live where your readers are. Markdown-native audience → Mermaid. Deep UML / exotic diagram types → PlantUML. Mixed course work → both, from one repository, each as text.

How to proceed: the workflow

Drawn once, maintained forever, the diagram lives as text in your repo



- 1 Pick the diagram type from the UML question you must answer (structure? behaviour? scope?)
- 2 Draft in the online editor, fast loop, no setup; steal a template from the Examples menu or course Loop
- 3 Save the source as text in the repo: diagram.puml or diagram.mmd (or a fenced block in README.md)
- 4 Commit & review: the diff shows exactly what changed in the model, reviewers read text, not pixels
- 5 Export SVG into the report/slides; keep the source file beside it
- 6 Model changes? Edit the text, re-render. You never touch a pixel again.

Your turn: Detective Game, episode II

SOLO · 7 MIN

Task A - sequence

Detective asks Lab to analyse a fingerprint.

Lab replies: match found.

Detective books the Suspect.

Add a note over Detective: case closed.

Number the messages.

Task B - classes

Suspect inherits from Person (name, alibi).

A Case is composed of many Clue objects.

Detective (a Person) solves at most one Case.

Show visibility (+/-) on every member.

Self-check before you swap: wrapper/first-line present · every label quoted where needed · include/extend dashed with stereotype · multiplicities in quotes · renders with no error image. **Stretch:** add a use-case diagram where (Provide Alibi) extends (Answer Questions).

Pro habits & classic pitfalls

What separates a diagram that survives the semester from one that does not

- **One diagram = one file**, named like code: checkout_sequence.puml, domain_model.mmd
- **Prefer SVG exports** in reports: scales to any size, diffs stay tiny
- **Quote labels** with spaces or () in both languages - prevents half of all parse errors
- **Comment your sources** (' in PlantUML, %% in Mermaid): future, you is a stranger
- **Do not fight the layout engine**: declare order, use aliases and direction hints, and if a diagram reads badly, suspect the MODEL, not the tool
- **Render in CI before deadline night**, not during it

A confused diagram is often a confused design. The renderer is your first design reviewer.

Five things to walk out with

- 1 A diagram can be a text file - so it can be versioned, diffed, reviewed and regenerated.
- 2 PlantUML: wrapper tags (@startuml / @enduml), deepest UML coverage, runs on a server or as a jar.
- 3 Mermaid: first-line keyword, instant browser preview, native in GitHub/GitLab Markdown.
- 4 Both editors are free, online, and shareable by URL - your workspace from any machine.
- 5 The workflow: editor → .puml/.mmd in git → SVG in report → edit text forever after.

Appendix A - PlantUML cheat sheet

Keep this slide; you will grep it all semester

```
@startuml / @endjson / @startgantt / @startmindmap ... ' wrappers
' comment line
actor A | participant P | class C | usecase "U" as u
A -> B : msg      A --> B : reply      autonumber
activate B / deactivate B
class C { +pub -priv #prot ~pkg  member: Type }
A -- B | A <|-- B | A *-- B | A o-- B | A ..> B | A ..|> B
"1" -- "*"          ' multiplicities in quotes
A .> B : <<include>>  A .> B : <<extend>>  ' dashed!
left to right direction  !theme plain  scale 2
```

Error image says 'line n'? Go to line n. Usually: missing @, unquoted label with spaces, or a stereotype typo.

Server URLs: /png/ /svg/ /txt/ + encoded text = shareable output links.

Local: java -jar plantuml.jar file.puml (some layouts use Graphviz; the Smetana engine avoids that dependency).

Appendix B - Mermaid cheat sheet

Same idea, different punctuation

```
flowchart TD|LR    sequenceDiagram    classDiagram
stateDiagram-v2    erDiagram    pie    gantt    gitGraph
%% comment line
A[text] rect    A(text) rounded    A{text} diamond
A([text]) stadium    A((text)) circle    A[(text)] cylinder
A --> B    A --- B    A -.-> B    A ==> B
A -- label --> B    A -->|label| B
participant P    actor A    P->>Q: msg    P-->>Q: reply
Note over P,Q: text
class C { +member: Type    +method(x: T) R }
C <|-- D    C *-- D    C o-- D    C --> D    C ..> D
[*] --> State1    State1 --> State2 : event
```

Parse error? Check the three classics: unknown arrow token, unquoted special character in a label, unclosed bracket.

In Markdown: fence the code with ```mermaid - GitHub and GitLab render it.

Headless/CI: `npx @mermaid-js/mermaid-cli -i in.mmd -o out.svg`

@startuml
skin rose
title OOAD example architecture diagram

skinparam handwritten true
skinparam shadowing true
skinparam BackgroundColor transparent
skinparam sequenceArrowColor #ff5733ff
skinparam noteBackgroundColor #07e84aff
skinparam noteBorderColor #d50de7ff
skinparam noteFontColor #000000ff

actor User
actor Customer

participant System
participant Webserver
participant Database
participant API

User -> System: Initiate Process
System -> Database: **Query** Data
System -> API: API request
API --> Database: Fetch Data

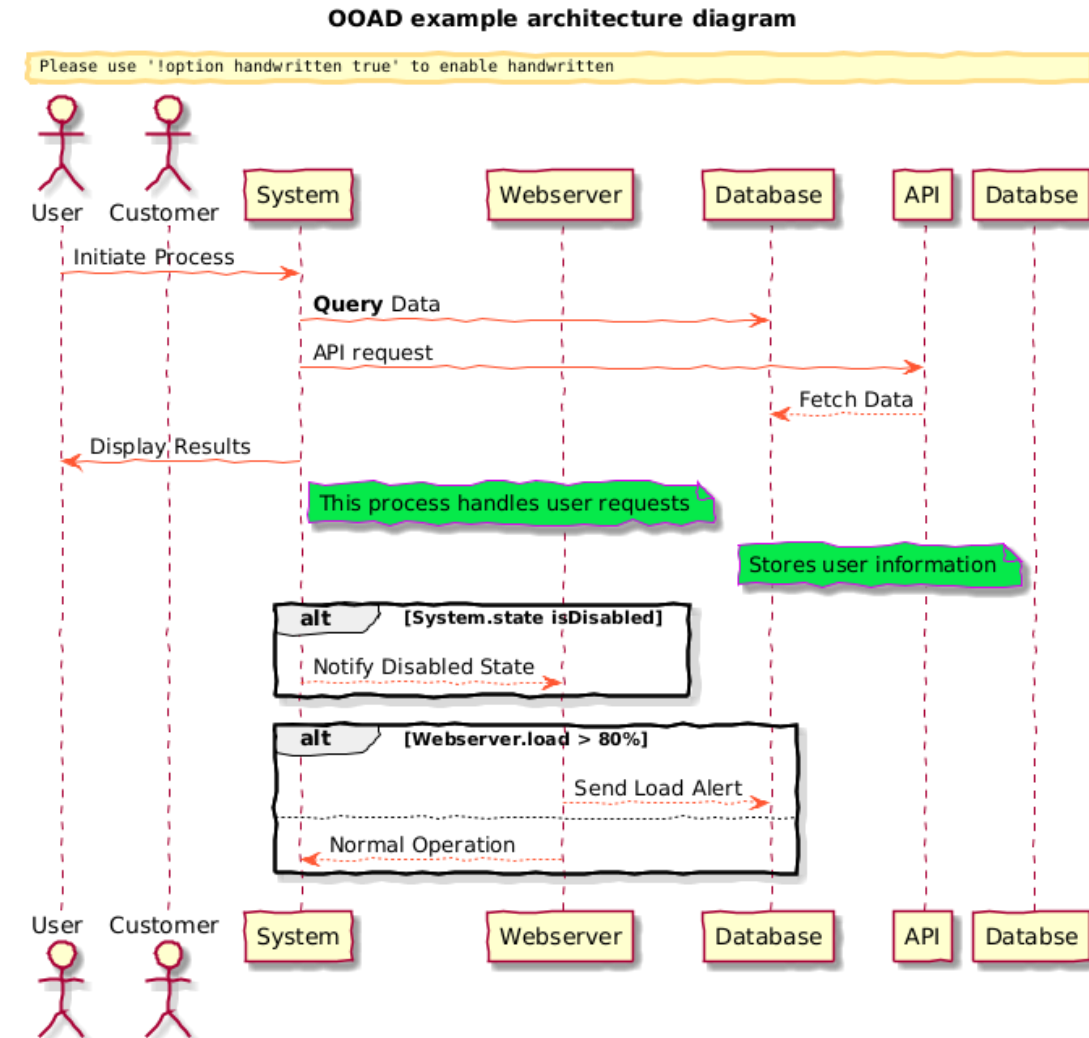
System -> User: Display Results
note right of System: This process handles user requests
note left of Database: Stores user information

alt System.state isDisabled
System --> Webserver: Notify Disabled State
end alt

alt Webserver.load > 80%
Webserver --> Database: Send Load Alert
else
Webserver --> System: Normal Operation
end alt

@enduml

BONUS (Customization)



@startuml

```
skinparam RoundCorner 5
skinparam ArrowFontSize 12
skinparam ClassBorderColor Black
skinparam ClassBorderColor<<custom>> Blue
skinparam Padding 1
skinparam linetype ortho
hide circle
hide stereotype
```

title Entity Model

footer Generated %date("yyyy-MM-dd") by UML Diagram Generator for OOAD Class by Sunder Ali Khowaja

```
entity Account <<standard>>
{
--
*AccountName (PN): String
PrimaryContact: Lookup
}
```

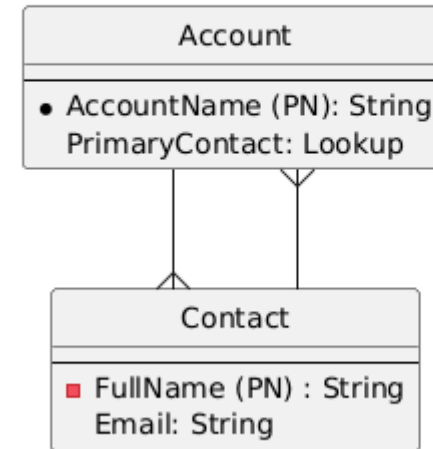
```
entity Contact <<standard>>
{
--
-FullName (PN) : String
Email: String
}
```

```
Contact --{ Account
Account --{ Contact
@enduml
```

BONUS (Customization)

Entity Model

Please use CSS style instead of skinparam padding



Generated 2026-09-09 by UML Diagram Generator for OOAD Class by Sunder Ali Khowaja